HDOC Day-70 Activity

Digital Root Using Recursion and a Static Call Counter

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-70

Aim

To write a recursive C program that calculates the **digital root** of a non-negative integer and uses a **static variable** to count the number of calls made to the recursive digital-root function.

What is a Digital Root?

The digital root of a number is obtained by repeatedly adding its digits until only one digit remains. For example: **9875 -> 29 -> 11 -> 2**, so the digital root is 2.

Problem Requirements

- Read a non-negative integer from the user.
- Calculate its digital root recursively.
- Repeatedly find the sum of its digits until the value becomes a single digit.
- Use a **static** variable to count recursive calls.
- Display both the digital root and the number of recursive calls.

Recursive Approach

The program first calculates the sum of digits. If that sum is already a single digit, it is the digital root. Otherwise, the function calls itself again using the new sum. A static variable inside the recursive function is incremented on every invocation and retains its value between recursive calls.

Algorithm

1. Read the number n.
2. Call **digital_root(n)**.
3. Inside digital_root(), declare a static call counter and increment it.
4. If n is a single digit, return n.
5. Otherwise calculate the sum of all digits of n.
6. Recursively call digital_root(sum).
7. Continue until a single-digit value is reached.
8. Display the returned digital root and the total call count.

C Program

```
#include <stdio.h>

int get_call_count(int reset)
{
    static int count = 0;

    if (reset)
        count = 0;
    else
        count++;

    return count;
}

int digital_root(unsigned long long n)
{
    get_call_count(0);

    if (n < 10)
        return (int)n;

    unsigned long long sum = 0;

    while (n > 0)
    {
        sum = sum + n % 10;
        n = n / 10;
    }

    return digital_root(sum);
}

int main()
{
    unsigned long long n;

    printf("Enter a number: ");
    scanf("%llu", &n);

    get_call_count(1);

    int result = digital_root(n);

    printf("Digital root: %d\n", result);
    printf("Recursive calls: %d", get_call_count(2));

    return 0;
}
```

Static Variable Explanation

The variable **count** is declared with the **static** keyword inside get_call_count(). A static local variable is created only once and keeps its value between function calls. Therefore, every invocation of digital_root() can increase the same persistent counter.

Dry Run - Number 9875

Call	Number	Digit sum / action
1	9875	9 + 8 + 7 + 5 = 29
2	29	2 + 9 = 11

3	11	1 + 1 = 2
4	2	Single digit: return 2

Therefore, the digital root of **9875** is **2**. With the program's counting convention, digital_root() is invoked **4 times**, including the final base-case call for 2.

Sample Test Cases

Input	Reduction	Digital Root
9875	9875 -> 29 -> 11 -> 2	2
1234	1234 -> 10 -> 1	1
99	99 -> 18 -> 9	9
5	Already one digit	5
0	Already one digit	0

Base Case

The condition **n < 10** is the base case because any value from 0 to 9 is already a single digit. At this point no further digit summing is required.

Complexity

Time Complexity: O(d) for practical integer input, where d is the number of digits processed across the reductions. The first digit-sum pass dominates because later values are much smaller.

Recursion Space: O(r), where r is the number of digital-root reduction levels.

Result

The program successfully calculates the digital root by repeatedly summing digits and recursively processing the result until one digit remains. A static variable is used to retain and report the recursive-call count.

Execution Example

```
Enter a number: 9875
Digital root: 2
Recursive calls: 4
```